

Resources Design

Módulo `resources` — Catálogo de contenido de estudio

Cómo el sistema enriquece cada unit del plan con material externo (videos, guías, sandboxes) sin alucinar URLs.

Qué resuelve

El `content` y los `exercises` generados por el LLM cubren lo que el alumno lee y practica formalmente. Falta el material complementario que pide el modelo 5P: videos cortos, sandboxes interactivos, documentación de referencia. Estos son recursos externos — el sistema los referencia, no los aloja.

Cómo funciona

Tabla `resources` con clave funcional `(skill_name, phase)`. Cuando se genera un plan, por cada (skill, fase) se ejecuta `resources_service.get_or_suggest`:

1. Si hay rows activas en BD → reutilizar.
2. Si no hay → pedir al LLM activo, persistir, devolver.

Resultado: cada combinación skill/fase paga UNA llamada al LLM en toda su vida. Dos planes con la misma skill comparten los mismos recursos.

Los recursos se embeben en el `plan_json` al guardar el plan, igual que los exercises. Quedan congelados con el plan.

Anti-alucinación de URLs

Los LLMs inventan URLs de YouTube. Para evitarlo:

- Al LLM le pedimos **título + canal/fuente + términos de búsqueda**, no URL exacta.
- Para docs oficiales estables hay una whitelist (`_KNOWN_DOC_DOMAINS` en `suggester.py`: MDN, react.dev, git-scm.com, freecodecamp.org, etc.). Si el LLM devuelve una `canonical_url` que matchea esos dominios, la usamos.
- Para todo lo demás (especialmente videos) armamos URL de búsqueda: `https://www.youtube.com/results?search_query=...`. Esos links nunca mueren — el alumno hace un click extra a la página de resultados.

Cantidades por fase

Fase	Cuántos	Tipos
<code>pasion</code>	2	video motivacional + lectura introductoria
<code>play</code>	2	sandbox interactivo + tutorial corto
<code>practica</code>	3	guía oficial + cheatsheet + plataforma de ejercicios

Tabla

```
CREATE TABLE resources (  
  id SERIAL PRIMARY KEY,  
  skill_name VARCHAR(255) NOT NULL,  
  phase VARCHAR(20) NOT NULL,  
  type VARCHAR(20) NOT NULL,          -- video | guide | sandbox | reading  
  title VARCHAR(500) NOT NULL,  
  url TEXT NOT NULL,  
  description TEXT,  
  duration_minutes INTEGER,  
  source VARCHAR(100),  
  language VARCHAR(5) DEFAULT 'es',  
  level VARCHAR(20),                  -- beginner | intermediate | advanced  
  generated_by VARCHAR(20) DEFAULT 'manual',  
  is_active BOOLEAN DEFAULT TRUE,  
  created_at TIMESTAMPTZ DEFAULT NOW()  
);
```

`is_active` permite desactivar un recurso sin borrarlo (preserva auditoría).

Manejo de fallas

Falla	Comportamiento
LLM no configurado	<code>suggester</code> devuelve <code>[]</code> , las units quedan sin <code>resources</code>
LLM falla (cuota, timeout)	<code>[]</code> , las units quedan sin <code>resources</code>
JSON malformado del LLM	Items inválidos se skippean

El POST nunca falla por culpa de los recursos. Si no hay, el alumno ve el plan sin material extra.

Cómo regenerar recursos de una skill

```
DELETE FROM resources WHERE skill_name = 'JavaScript';
```

El próximo plan con esa skill vuelve a llamar al LLM.

Lo que no está

- Endpoints CRUD para administrar el catálogo. Para curar manualmente hoy se usa SQL directo. Cuando haga falta, sumar `app/modules/resources/router.py` (el service y repository ya están listos).
- Constraint UNIQUE `(skill_name, phase, url)`. En alta concurrencia podría duplicar. No bloquea ni rompe; suma rows.
- TTL del cache. Los recursos quedan vigentes para siempre hasta borrado manual.